AI-Based Biometric Evidence Authentication for Post-Quantum Digital Identity Systems

Course Project Report

Name	Registration number	Gmail
Aviral Bhan	23BIT0241	aviralbhanmv@gmail.com
Krish Manoj Mehta	23BIT407	[Gmail address to be added]
Abhijay Singh Panwar	23BIT0353	[Gmail address to be added]

Supervisors: Dr. Aswani Kumar Cherukuri and Dr. Rajasekhara Reddy D.

Vellore Institute of Technology, Vellore

October 2026

Abstract

Face-based digital identity systems now approve logins, payments and account recovery from a few seconds of selfie video, so the evidence they receive must be both genuine and protected. Two threats meet here: spoofed evidence (printed photos, screen replays, morphed enrolment photos, deepfakes and virtual-camera injection) and future quantum computers that could break the RSA and elliptic-curve cryptography that protects the evidence today. Existing work usually handles one threat at a time: post-quantum schemes protect templates or channels, while attack detectors produce scores that stay unsigned inside the server. This project builds and measures a working prototype that does both and links them. Every transaction receives a single-use challenge with a 32-byte nonce valid for 60 seconds, a hash of the transaction and two random liveness actions. The client records a four-second video, signs a manifest with ML-DSA-65 (FIPS 204) and encrypts everything with AES-256-GCM under a key agreed through ML-KEM-768 (FIPS 203). The server runs six protocol checks in a fixed order and then a face matcher and five detectors in parallel: presentation attacks (M1), morphing (M2), deepfakes (M3), replay through liveness actions (M4) and virtual-camera injection (M5). The novelty is that the final verdict, with its scores, model versions, thresholds, decision and the hashes of the challenge, transaction and video, is itself signed with ML-DSA-65, so the AI decision becomes evidence that can be re-checked later. A classical mode (X25519 and Ed25519) runs the same code, so the cost of post-quantum cryptography is measured with the detectors held constant. On held-out subjects of the FRLL face set the ArcFace matcher rejected none of 255 genuine pairs at a false-match rate of 0.10%. The differential morph detector let through at most 0.37% of morphs from five morphing tools while rejecting 0.59% of genuine pairs. LayerNorm-only tuning of the GenD deepfake detector raised the AUC on the independent OpenForensics Test-Dev set from 0.943 to 0.998 and cut the share of fakes accepted at a 0.5% genuine-rejection

target from 65.2% to 11.5%. Each ML-KEM-768 and ML-DSA-65 operation took at most 0.473 ms (median) on a laptop CPU, and PQ mode adds 4,301 bytes of key-exchange and signature data per transaction. End to end, a transaction took a median of 3205 ms in classical mode and 3172 ms in PQ mode, a difference smaller than the run-to-run variation of the detectors; all cryptographic steps together took 23.2 ms and 26.4 ms, 0.73% and 0.83% of the total. The 1,000-run replay suite with the real models had not finished when this report was generated. With an enrolment made in the same room, the real user was accepted in 18 of 20 live webcam captures across both modes. Fine-tuning M1 on CelebA-Spoof and training the M5 classifier on recorded virtual-camera sessions are still pending and are reported as such. The results show that post-quantum cryptography is affordable for biometric evidence but only proves where evidence came from and that it was not changed; the detectors, and the signed verdict that records their decision, are what show it is genuine.

Keywords: Deepfake detection; Face anti-spoofing; ML-DSA; ML-KEM; Morphing attack detection; Post-quantum cryptography.

1. Introduction

Remote face verification has become the default way to prove identity online. Banks, payment apps and government services ask the user to look into a phone or laptop camera, and a server decides from a few seconds of video whether the person is who they claim to be. Because the decision is made far from the user, the server can only trust what it receives, and attackers have learned to feed it convincing fakes: printed photos and replayed screens held up to the camera (presentation attacks [1]), enrolment photos that blend two faces so both people match (morphing [2]), AI-generated or face-swapped video (deepfakes [3]), and fake video streams fed in through virtual-camera software so that no physical camera is involved at all (injection [4]).

At the same time the cryptography that protects this evidence is reaching the end of its life. Shor's algorithm factors integers and computes discrete logarithms in polynomial time on a large quantum computer [5], which breaks RSA, Diffie–Hellman and elliptic-curve schemes such as X25519 and Ed25519; Grover's search gives a quadratic speed-up against symmetric keys [6], which is why 256-bit keys are now preferred. A recent survey summarises how fast the hardware is improving [7]. The danger is not only in the future: an attacker can record encrypted traffic today and decrypt it once a capable machine exists ("harvest now, decrypt later") [8]. For biometrics this is worse than for passwords, because a face cannot be reissued after it leaks [9]. NIST therefore standardised ML-KEM for key establishment and ML-DSA for signatures in August 2024 [10], [11].

Industry is starting to combine the two worlds. Daon describes a patented "Quantum Identity" architecture that binds biometric evidence to individual transactions [12], [13], and authID signs biometric transactions with ML-DSA-65 [14]. Academic work protects face templates with lattice-based or code-based encryption [15], [16], [17]. Post-quantum cryptography, however, works like a tamper-proof envelope: it proves that nobody opened or swapped the envelope, even after quantum computers exist, but it says nothing about whether the photo inside is a printout, a morph or a deepfake. A secure identity system needs both properties at once, and it needs a record of why each decision was made.

1.1 Problem statement and why post-quantum security

The problem addressed is: *how can a remote face-verification system accept only genuine, live, fresh evidence for one specific transaction, keep that evidence confidential and unforgeable against an adversary with a future quantum computer, and leave an auditable, tamper-evident record of the AI decision, at a cost that users will not notice?* Post-quantum cryptography is needed because biometric evidence has a long shelf life: a face video captured today identifies the same person for decades, so it must stay secret beyond the expected arrival of cryptographically relevant quantum computers. Quantum computing is not used to run the system; the project uses quantum-resistant algorithms on ordinary hardware, which is how post-quantum security is deployed in practice.

1.2 Goals

- Protect the evidence in transit with ML-KEM-768 and ML-DSA-65, and bind it to one transaction and one challenge so that it cannot be replayed or reused for another payment.
- Detect five attack classes (presentation, morphing, deepfake, replay, injection) with open models on openly downloadable datasets, reported per attack type at three fixed operating points.
- Sign the final verdict (scores, model versions, thresholds, decision and hashes) so that the AI decision can be re-checked later.
- Measure accuracy, latency, CPU, memory and bytes per request in classical and post-quantum modes with identical detectors, to quantify the cost of post-quantum security.
- Keep the prototype reproducible: public code, pinned versions, scripts for every result and no hand-typed numbers.

1.3 Novelty

Neither post-quantum protection of biometric data nor AI attack detection is new, and this report does not claim either. To our knowledge, in the open literature, existing systems treat the detector's verdict as an unsigned internal result. Here the verdict itself becomes post-quantum-signed evidence: the server signs, with ML-DSA-65, one record that binds the challenge, the transaction hash, the video hash, every detector score, the model versions, the threshold version and the decision. Anyone holding the server's public key can later prove that a stored decision was not edited and that it belongs to that exact transaction and video.

The second contribution is empirical: one testbed in which the same detectors run under classical and post-quantum cryptography, so that the report can state, with measurements, what post-quantum cryptography adds (confidentiality against future quantum attackers, at a measured cost in time and bytes) and what it does not (it does not stop a single presentation, morph or deepfake attack, which the AI modules must catch).

1.4 Organisation of the report

Section 2 reviews related work, the theory behind the quantum threat and lattice cryptography, and the gap this project fills. Section 3 formulates the problem and describes the protocol, the six server checks, the five detectors and the implementation. Section 4 gives the experimental setup, the metrics and the results, comparing classical and post-quantum cryptography and discussing sources of error. Section 5 discusses novelty, insights and limitations, and Section 6 concludes. The code link, originality and AI-usage statements, references and an appendix follow.

2. Background and Related Work

2.1 Literature review

Post-quantum protection of biometric data. Román *et al.* combined the Kyber and Saber public-key schemes with homomorphic encryption so that face templates can be compared

while encrypted [15]. Arjona *et al.* built a similar scheme on Classic McEliece and showed real-time matching of protected FaceNet embeddings [16]. Olushola and Meenakshi fused face, iris and fingerprint features and matched them under CKKS homomorphic encryption with ML-KEM (Kyber-1024) session keys [17]. Gowri Ganesh *et al.* sketched, in simulation, a pipeline that places a CNN presentation-attack detector next to ML-KEM [18]. These works protect templates and channels; none signs the detector's decision, and none measures replay, morphing, deepfake and injection attacks in one system.

Industry systems. Daon's five-patent architecture checks whether evidence is real, live and transaction-bound; its U.S. Patent 12,695,590 adds a transaction-specific noise vector to biometric features to resist injection and replay [12]. A trade report notes that post-quantum identity is moving "beyond cryptography" to the evidence itself [13], and authID now offers ML-DSA-65 signatures on biometric transactions [14]. Veridos argues that biometric records need the earliest post-quantum attention because they cannot be revoked [9]. None of these publishes measurements or signs the AI verdict as an auditable record.

Attack detection. Presentation attack detection (PAD) is evaluated with the APCER/BPCER metrics of ISO/IEC 30107-3 [1]; CelebA-Spoof is a large open PAD dataset [19], and the MiniFASNet models of Silent-Face-Anti-Spoofing are light enough for real time [20]. Challenge-response liveness, in which the user must react to something unpredictable, appears in Face Flashing (random screen colours) [21] and GOTCHA (challenges that degrade real-time deepfakes) [22]. Morphing attacks were introduced by Ferrara *et al.* [2]; differential morphing attack detection (D-MAD) compares the enrolled photo with a trusted live capture through deep face embeddings [23], and FRLL-Morphs provides morphs from several tools [24], [25]. For deepfakes, OpenForensics offers multi-face forgeries in the wild [26], and GenD adapts a CLIP vision encoder [27] by tuning only its LayerNorm parameters [28]. Virtual-camera injection can be detected from capture metadata and timing [4].

Table 1. Comparison of the closest related work with this project.

Work	PQ crypto	Attack detection	Transaction binding	Signed AI verdict	Measured cost
Román *et al.* 2022 [15]	Kyber, Saber	No	No	No	Template ops
Arjona *et al.* 2023 [16]	McEliece	No	No	No	Template ops
Olushola & Meenakshi 2026 [17]	ML-KEM-1024	Discussed	No	No	Matching latency
Gowri Ganesh *et al.* 2026 [18]	ML-KEM	PAD (simulated)	No	No	Feasibility only
Daon patents 2026 [12]	Yes (claimed)	Injection, replay	Yes	No	Not published
authID 2026 [14]	ML-DSA-65	Liveness (product)	Yes	Not stated	Not published
Face Flashing, GOTCHA [21], [22]	No	Liveness	Challenge only	No	Detection only
This project	ML-KEM-768, ML-DSA-65	M1–M5 + matcher	Yes (checks 1–6)	**Yes**	Classical vs PQ

Table 1 shows that each column has been covered by some earlier work, but no row other than this project covers all of them, and none signs the AI verdict.

2.2 Theoretical framework

The quantum threat. Shor's algorithm finds the period of a modular function with the quantum Fourier transform, which yields the factors of an RSA modulus or the discrete logarithm behind Diffie–Hellman and elliptic-curve keys in polynomial time [5]. X25519 [29] and Ed25519 [30], used here as the classical baseline, fall to it. Grover's algorithm searches an unstructured space of size N in about $\sqrt{N}$ steps [6], so a k-bit symmetric key keeps about k/2 bits of security; AES-256 and SHA-384 therefore keep a comfortable margin, and they are used for all symmetric work in this project [31], [32].

Learning with errors. The post-quantum schemes used here rest on the learning-with-errors (LWE) problem [33]: given a random matrix A and $b = A \cdot s + e \pmod q$, where s is secret and e is a small random error, recover s. The small error makes the linear system hard to solve, and no efficient quantum algorithm is known. Module-LWE replaces integers by polynomials in the ring $R_q = Z_q[X]/(X^{256} + 1)$ and uses small k×k matrices of such polynomials, which keeps keys small while retaining worst-case hardness guarantees [34]:

$$b = A \cdot s + e, \quad A \in R_q^{k \times k}, \quad s, e \text{ small}, \quad R_q = Z_q[X]/(X^{256} + 1) \qquad (1)$$

ML-KEM-768 (FIPS 203, derived from CRYSTALS-Kyber [35]) uses k = 3 and q = 3329 and reaches NIST security category 3. The public key is 1,184 bytes and the ciphertext 1,088 bytes; both sides obtain the same 32-byte shared secret, and a Fujisaki–Okamoto transform makes it secure against chosen-ciphertext attacks [10]. **ML-DSA-65** (FIPS 204, from CRYSTALS-Dilithium [36]) uses a 6×5 module over q = 8,380,417 and the Fiat–Shamir-with-aborts technique; at category 3 its public key is 1,952 bytes and a signature 3,309 bytes [11]. Both were checked against NIST ACVP known-answer vectors [37].

Symmetric layer. The shared secret is turned into an AES key with HKDF [38], built on HMAC [39]:

$$PRK = \text{HMAC-SHA384}(\text{nonce}, ss) \qquad (2)$$

$$K = \text{HMAC-SHA384}(PRK, \text{info} \,\|\, 0x01)[0 \ldots 31] \qquad (3)$$

with the nonce as the HKDF salt, and where info = "BEA-v1|" ‖ mode ‖ "|" ‖ challenge_id. AES-256-GCM [32] then encrypts and authenticates the evidence with a random 96-bit IV and a 128-bit tag, using AAD = mode ‖ "|" ‖ challenge_id. Changing any byte of the ciphertext, the IV, the KEM ciphertext, the mode or the challenge id changes either K or the tag, so decryption fails.

Biometric error rates. Following ISO/IEC 30107-3 [1], for a detector with score s and threshold τ (higher score = more likely an attack):

$$APCER = |\{\text{attacks with } s \le \tau\}| \,/\, |\text{attacks}| \qquad (4)$$

$$BPCER = |\{\text{bona fide samples with } s > \tau\}| \,/\, |\text{bona fide samples}| \qquad (5)$$

APCER is reported separately for each attack type. The equal error rate (EER) is the point where APCER = BPCER, and the AUC is the probability that a random attack scores above a random bona fide sample. For the matcher, the false match rate (FMR) is the share of impostor pairs whose similarity reaches the threshold and the false non-match rate (FNMR) the share of genuine pairs below it.

2.3 Gap analysis

- **Crypto without detection.** Post-quantum biometric schemes protect templates and channels, but a perfectly encrypted deepfake is still a deepfake [15], [16], [17].
- **Detection without protected evidence.** Detector papers assume the frames reach the model unchanged; they do not bind the sample to a transaction or a challenge.
- **Unsigned decisions.** In every system reviewed, the detector's score and the final decision are internal values that an insider could later change without trace.
- **No measured trade-off.** No reviewed work measures, in one pipeline with real detectors, how much post-quantum cryptography adds to latency and bytes, or which attacks it does and does not stop.
- **Optimistic thresholds.** Thresholds are often chosen on the test data; here every threshold is set on data separate from the data it is reported on.

This project fills these gaps with a single prototype that binds, encrypts and signs the evidence, runs five detectors, signs the verdict and measures both modes under identical conditions.

3. Proposed Methodology and System Architecture

3.1 Problem formulation

A user U has enrolled a device signing key pk_{dev} and a face template t. For a transaction tx_context, the server issues a challenge C = (id, n, h_{tx}, a_1, a_2, mode, expiry), where n is a 32-byte nonce, h_{tx} = SHA-384(canonical_json(tx_context)) and a_1, a_2 are two liveness actions. The client returns a sealed bundle B carrying a video v, capture metadata m, a manifest M = (mode, id, n, h_{tx}, SHA-384(v), timestamp) and a signature σ = Sign(sk_{dev}, SHA-384(M)). The server must output accept only if:

$$Check_i(B, C) \text{ passes } (i = 1 \ldots 6) \ \wedge \ s_k(v, m) \leq \tau_k \ (k = M1 \ldots M5) \ \wedge \ \cos(e(v), t) \geq \tau_{match} \quad (6)$$

and it must sign the record R = (id, mode, h_{tx}, SHA-384(v), scores, model versions, threshold version, decision, reason code, time) as ρ = Sign(sk_{server}, SHA-384(canonical_json(R))). Each threshold τ_k is the smallest value whose bona fide rejection on separate calibration data is at most β = 0.5%; with five modules this keeps the combined false rejection near 2.5%. The security goals are freshness (no reuse of a challenge), binding (evidence valid for one transaction only), confidentiality including against harvest-now-decrypt-later, integrity and origin authentication, genuineness of the face, and auditability of the decision. The threat model is summarised in Table 2.

Table 2. Threat model: attacks considered and the component that counters each.

Attack	What the attacker does	Countered by
Harvest now, decrypt later	Records traffic to decrypt with a future quantum computer	ML-KEM-768 + HKDF-SHA384 + AES-256-GCM
Tampering in transit	Changes bytes of the sealed bundle	Check 3 (GCM tag), 5 (hash), 6 (signature)
Replay / delayed use	Sends a captured bundle again or later	Check 1 (single use, 60 s TTL)
Re-targeting	Re-uses evidence for another payment	Check 4 (signed nonce and tx_hash)
Downgrade	Relabels a PQ challenge as classical	Check 2; mode in HKDF info and GCM AAD
Forged device	Builds a bundle without the device key	Check 6 (ML-DSA-65 / Ed25519)
Key substitution	Gives the client a fake server key	Server KEM key pinned at enrolment
Wrong person	Someone else at the camera	Matcher (ArcFace, FMR 0.1%)
Photo, screen, mask	Presents an artefact to the camera	M1 (MiniFASNet), M4 (actions)
Pre-recorded video	Plays a recording of the user	M4: actions unknown before the nonce
Morphed enrolment	Enrols a face blended from two people	M2 (D-MAD)
Deepfake	Synthesised or face-swapped video	M3 (GenD)
Virtual-camera injection	Feeds a stream via OBS or v4l2loopback	M5 (rules + gradient-boosted trees)
Editing a verdict	Changes a stored score or decision	ML-DSA-65-signed verdict record

Out of scope, and stated as limits: a compromised server, side channels, denial of service, enrolment fraud, a strong attacker on a rooted device and high-quality 3D masks.

3.2 Algorithm and protocol design

Fig. 1 shows the four layers. Layer A captures the evidence, Layer B seals it with post-quantum cryptography, Layer C verifies it and Layer D measures everything. Arrows: (1) the server issues a challenge; (2) the client builds the manifest from the video; (3) the sealed bundle is sent to POST /verify; (4) the client receives only "accepted" or "rejected"; (5) stage timings and scores go to the evaluation layer.

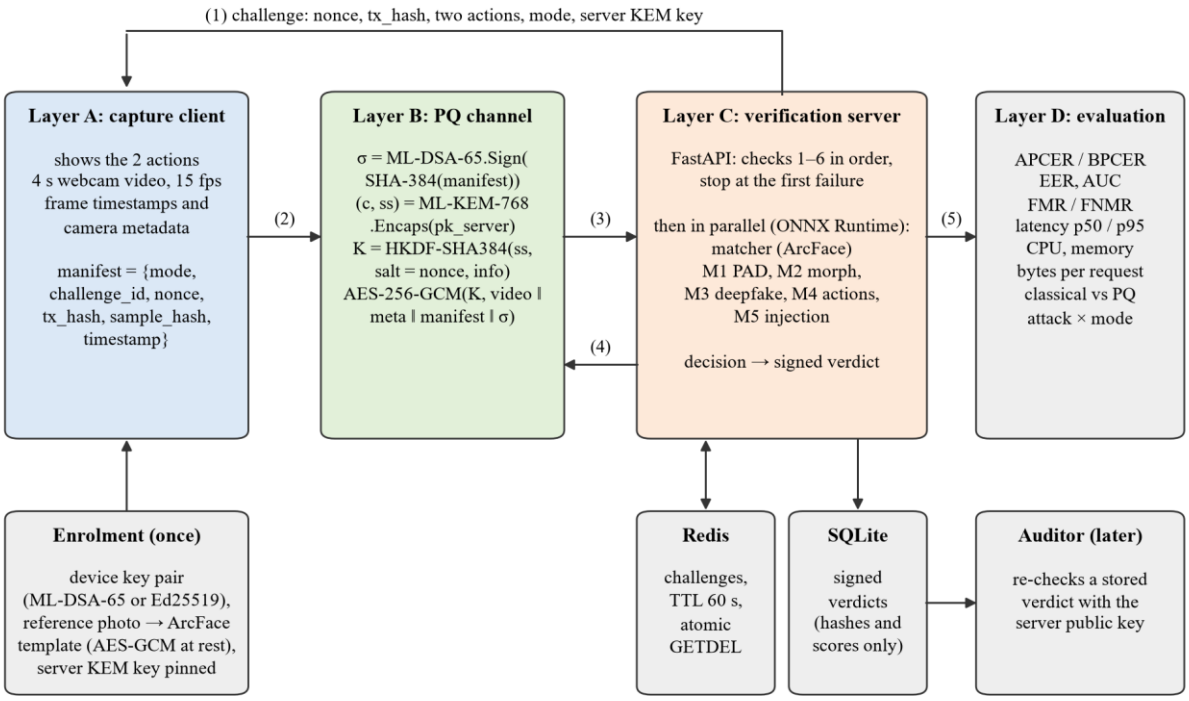


Fig. 1. System architecture: capture client, post-quantum channel, verification server and evaluation harness.

One transaction, step by step

1. **Challenge.** The server draws a 32-byte nonce from the OS random generator, computes tx_hash, chooses two actions and stores the challenge in Redis with a 60-second lifetime and the server KEM public key.
2. **Actions.** m = HMAC-SHA256(server_secret, nonce) drives a Fisher–Yates shuffle of {blink twice, turn left, turn right}: for i = n−1 … 1, swap item i with item m[n−1−i] mod (i+1); the first two items are the actions. Nobody can predict them before the nonce exists.
3. **Capture.** The client checks that the challenge names the pinned server key and the transaction it is showing, displays the actions and records a 4-second video with per-frame timestamps and camera metadata.
4. **Manifest and signature.** The client builds the manifest as canonical JSON (sorted keys, no spaces) and signs SHA-384(manifest) with its ML-DSA-65 device key.
5. **Sealing.** The client encapsulates to the server key, derives K with HKDF-SHA384 and encrypts video ‖ metadata ‖ manifest ‖ signature (each part length-prefixed) with AES-256-GCM.
6. **Verification.** The server runs checks 1–6 in order and stops at the first failure (Table 3, Fig. 2); then it runs the matcher and M1–M5 in parallel.
7. **Decision and verdict.** The decision rule below gives accept or reject, and the server signs and stores the verdict record. Exactly one verdict is stored per consumed challenge, rejections included.

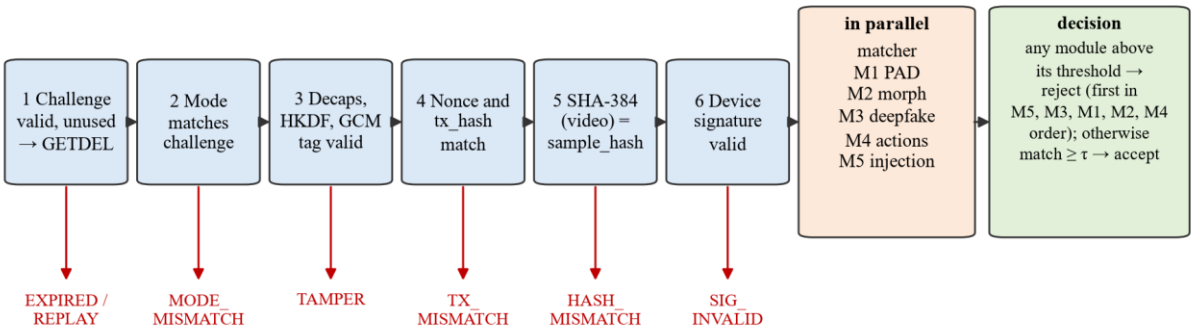


Fig. 2. Server verification pipeline: six protocol checks in a fixed order, each with its own reason code, followed by the detectors and the decision. Every consumed challenge gets one signed verdict, rejections included.

Table 3. Server checks, in their fixed order; the first failure stops verification.

#	Check	Reason code
1	Challenge exists, is not expired and not used; it is consumed atomically (Redis GETDEL)	EXPIRED / REPLAY
2	Mode of the bundle equals the mode stored with the challenge	MODE_MISMATCH
3	Decapsulate, derive K, AES-GCM tag valid	TAMPER
4	Manifest nonce, tx_hash, challenge id and mode equal the issued values	TX_MISMATCH
5	SHA-384(video) equals manifest.sample_hash	HASH_MISMATCH
6	Device signature valid under the enrolled device key	SIG_INVALID

The lookup runs first because the mode to compare against is stored with the challenge; a downgraded request therefore still consumes the challenge and cannot be retried. The client never sees the reason code, so an attacker cannot learn which check fired. Table 4 lists the detectors that run after the checks.

Table 4. Face matcher and detection modules M1–M5.

Module	Method	Score and rule	Reason code
Matcher	ArcFace ResNet-50 (InsightFace buffalo_l) [40], [41]; YuNet face detection [42]	cosine(e, t) ≥ 0.2192 (FMR 0.1%)	NO_MATCH
M1 PAD	MiniFASNetV2 (scale 2.7) + MiniFASNetV1SE (scale 4.0) [20]	s = 0.5 + ½(max(p0, p2) − p1); reject if s > 0.5	PAD_FAIL
M2 Morph	D-MAD: e(reference) − e(live) → RBF SVM [23], [43]	sigmoid(d) > 0.5608 (BPCER 0.5%)	MORPH_FLAG
M3 Deepfake	GenD, CLIP ViT-L/14, LayerNorm-tuned [27], [28]	mean P(fake) over 3 frames > 0.9182	DEEPFAKE_FAIL
M4 Replay	Checks 1–6 + MediaPipe blendshapes and head yaw [44]	both actions seen in order, else s = 1	SCRIPT_FAIL
M5 Injection	Virtual-camera name rule + HistGradientBoosting on 16 timing features [4], [43], [45]	name rule now; classifier pending	INJECTION_FAIL

Face pipeline. YuNet runs once per frame with a score threshold of 0.5; the face used is the largest among detections within 0.1 of the best score, and it must be at least 112 pixels wide. The five landmarks are aligned to the ArcFace template with a similarity transform, giving a 112×112 crop for ArcFace and M2, a 224×224 crop for GenD and 80×80 crops at two scales for MiniFASNet.

M2 morph detection. For an enrolled reference r and a live probe p the feature is the difference of their L2-normalised ArcFace embeddings, standardised and classified by an RBF support vector machine with balanced class weights:

$$f = e(r) - e(p), \quad d(f) = \Sigma_i \, \alpha_i \, y_i \, exp(-\gamma \|f_i - f\|^2) + b, \quad s_{M2} = 1 / (1 + e^{-d(f)}) \qquad (7)$$

A genuine pair gives a small, typical difference; a morphed reference only half-matches the live person and gives a different one.

M3 deepfake detection. Each 224×224 crop is normalised with CLIP's mean and standard deviation, the pooled feature h is L2-normalised and a linear head gives P(fake) = softmax(W·h/‖h‖ + b)_fake. Tuning trains only the scale γ and shift β of each LayerNorm, $LN(x) = \gamma \odot (x - \mu)/\sqrt{(\sigma^2 + \varepsilon)} + \beta$, i.e. 102,400 of about 304 million parameters, with class-weighted cross-entropy and the AdamW optimiser [46].

M4 liveness actions. Per frame, the blink level is b = ½(eyeBlinkLeft + eyeBlinkRight) and the head yaw is yaw = (180/π)·atan2(R_{02}, R_{22}) from the face rotation matrix. A blink is counted when b rises above 0.5 and then falls below 0.3 (hysteresis); a turn is counted when |yaw| > 15° and reset below 7.5°. A recording cannot follow actions that did not exist when it was made, and the video hash is signed together with the nonce that chose them.

M5 injection. A camera name matching a known virtual camera (OBS Virtual Camera, v4l2loopback, ManyCam and others) scores 1. The planned classifier uses 16 features of the capture: mean, standard deviation, coefficient of variation and 5th/95th percentiles of the inter-frame intervals $\Delta t_i = t_{i+1} - t_i$, the share of intervals within 1 ms of the median, the duplicate-frame ratio, requested versus delivered resolution and frame rate, and the latency of a resolution change. Virtual cameras tend to be too regular, repeat frames and switch resolution instantly.

Decision rule. Reject if any module score exceeds its threshold, reporting the first in the order M5, M3, M1, M2, M4; otherwise accept if and only if the match similarity reaches τ_{match}. **Signed verdict.** The server builds the record R above, signs SHA-384(canonical_json(R)) with ML-DSA-65 in PQ mode (Ed25519 in classical mode) and stores R and the signature in SQLite. Only hashes and scores are stored, never face images, and scripts/verify_verdict.py re-checks any stored verdict with the server's public key.

3.3 Implementation details

The prototype is written in Python with type hints and a pytest suite for every module. Table 5 lists the frameworks and the hardware. No quantum hardware or simulator is used: post-quantum algorithms are designed to run on classical computers, and the "backend" for all cryptography is the OpenSSL-based Python *cryptography* library [47]. Neural models are

exported to ONNX and served with ONNX Runtime on the CPU [48]; the GPU was used only to tune M3.

Table 5. Software frameworks and hardware.

Component	Choice (version)
Language, OS	Python 3.12.10, Windows 11 (development and measurements)
Cryptography	cryptography 50.0.2 (OpenSSL): ML-KEM-768, ML-DSA-65, X25519, Ed25519, HKDF, AES-GCM
Server	FastAPI 0.143, Uvicorn 0.54, Redis protocol (fakeredis 2.39 locally; redis:7 in Docker), SQLite
Inference	ONNX Runtime 1.31 (CPU), OpenCV YuNet, MediaPipe 1.1 Face Landmarker
Training / export	PyTorch 2.14 (CUDA 13), transformers 4.56 for GenD export, scikit-learn 1.9
Evaluation	NumPy 2.5, psutil 7.2, locust 2.46, matplotlib
CPU	Intel Core i7-13700HX (24 logical processors), 32 GB RAM
GPU	NVIDIA GeForce RTX 4050 Laptop GPU, 6 GB (M3 tuning only)
Quantum backend	None; not applicable (post-quantum algorithms run on classical hardware)

The repository separates crypto/ (the CryptoSuite for both modes and the canonical manifest), server/ (challenge store, checks, decision, verdict, parallel pipeline), client/ (enrolment, capture, sealing), preprocessing/, detectors/ (one package per module), eval/ (metrics, harness, performance and attack-matrix scripts) and scripts/ (downloads, training, calibration, replay suite, verdict verification). All thresholds and paths live in config.yaml, and every number in this report is read from a CSV in eval/results/ by the script that builds the report.

4. Experimental Setup

4.1 Setup and parameters

The template for this report asks for qubits, shots and the optimiser. No quantum circuit is executed in this project, so qubits and shots do not apply; Table 6 lists the parameters that play their role here: security levels, protocol constants, thresholds, the optimiser used for tuning and the sizes of every experiment.

Table 6. Experimental parameters.

Parameter	Value
Qubits, shots	Not applicable (no quantum circuit; post-quantum algorithms on a classical CPU)
Security level	NIST category 3: ML-KEM-768, ML-DSA-65; AES-256-GCM; SHA-384; HKDF-SHA384
Classical baseline	X25519 (ephemeral client, static server), Ed25519; same symmetric layer
Challenge	Nonce 32 B, lifetime 60 s, single use (GETDEL); 2 of 3 actions
Capture	4 s video, 640×480, JPEG frames sampled at 15 fps
Face detection	YuNet score ≥ 0.5, score margin 0.1, face width ≥ 112 px
Thresholds (version)	preliminary-val-2026-10-12-10537d5e: match 0.2192, M1 0.5, M2 0.5608, M3 0.9182, M4 0.5, M5 0.5

Parameter	Value
Optimiser (M3 tuning)	AdamW, learning rate 0.0001, batch 16, fp16, 2 epochs, 102,400 trainable parameters, class-weighted cross-entropy
M2 classifier	RBF SVM, class_weight = balanced; 5-fold GroupKFold by subject, thresholds from an inner 4-fold CV
M3 frames	3 sampled frames per request, scores averaged
Crypto benchmark	1,000 runs per operation and mode, 2 MiB payload for sealing
End-to-end run	500 genuine transactions per mode after 3 warm-up runs; real models; in-process server with fakeredis
Replay suite	1,000 runs per attack and mode, plus 1,000 genuine runs per mode
Attack × mode matrix	100 OpenForensics Test-Dev fake faces, 100 real faces, 100 FRLL-Morphs pairs (all five tools), 27 webcam sessions; both modes

Table 7 lists the data. All datasets are openly downloadable; none is redistributed with the code. Splits are by identity (or by source image for OpenForensics), so no person or source image is in both training and test data.

Table 7. Datasets and how each is used.

Dataset	Size used	Use and protocol
FRLL [49]	102 subjects, 1350×1350 (CC BY 4.0)	Matcher: enrol neutral front; probe smiling and four 3/4 views; subjects split into validation and test halves
FRLL-Morphs [24], [25]	AMSL 4,030, others 2,290 pairs each; 1,020 bona fide pairs	M2: 5-fold identity-grouped CV; thresholds from training subjects only
OpenForensics Val [26]	3,951 real / 9,861 fake faces ≥ 112 px	M3: tuning (80%) and threshold calibration (20%), split by source image
OpenForensics Test-Dev	10,468 real / 19,865 fake faces	M3: independent test, never used for tuning or thresholds
CelebA-Spoof [19]	Not yet downloaded	M1 fine-tuning and evaluation (pending)
Own webcam recordings	32 logged genuine captures; 27 saved sessions	Live acceptance and the bona fide side of the attack matrix (team members, with consent)

4.2 Expected outcomes and performance metrics

Table 8. Metrics and the target for each.

Metric	Definition	Expected outcome
APCER per attack type	Share of attacks accepted	As low as possible at the fixed BPCER
BPCER	Share of bona fide samples rejected	0.5% per module (also 1% and 5% reported)
EER, AUC	Error where APCER = BPCER; ranking quality	Low EER, AUC close to 1
FMR, FNMR	Matcher false match / non-match rate	FMR 0.1% at a low FNMR

Metric	Definition	Expected outcome
Replay rejection	Share of protocol attacks rejected with the right code	100%, with 0% false rejection of genuine runs
Latency p50 / p95	Per stage and end to end (perf_counter_ns)	Crypto a small share of the total
CPU %, peak RSS	Sampled every 50 ms with psutil	Similar in both modes
Bytes per request	Size of the sealed bundle	About +4.3 kB in PQ mode

Operating points are fixed in advance: every detector is reported at BPCER targets of 0.5%, 1% and 5%, and every threshold is computed on data separate from the data it is reported on. For a target β and n sorted bona fide calibration scores, $\tau = s_{(n - \lfloor \beta n \rfloor)}$, so at most $\lfloor \beta n \rfloor$ bona fide samples score above it.

4.3 Comparative analysis and discussion

4.3.1 Classical versus post-quantum cryptography

Table 9. Time per cryptographic operation in microseconds (1,000 runs each).

Operation	Classical p50	Classical p95	PQ p50	PQ p95
Key generation (KEM)	22.9	25.4	104.7	127.5
Encapsulate (client)	48.1	57.2	38.8	48.3
Decapsulate (server)	22.9	27.4	40.2	47.6
Signature key generation	22.4	24.8	117.5	144.9
Sign manifest (client)	22.9	26.0	473.1	1500.0
Verify manifest (server)	63.6	79.5	111.1	176.5
Sign verdict (server)	33.0	41.9	468.4	1461.0
HKDF-SHA384	3.9	4.6	3.8	4.1
Seal 2 MiB bundle (client)	1536.4	1712.6	1526.8	1710.5
Unseal 2 MiB bundle (server)	1076.3	1202.4	1090.0	1207.4

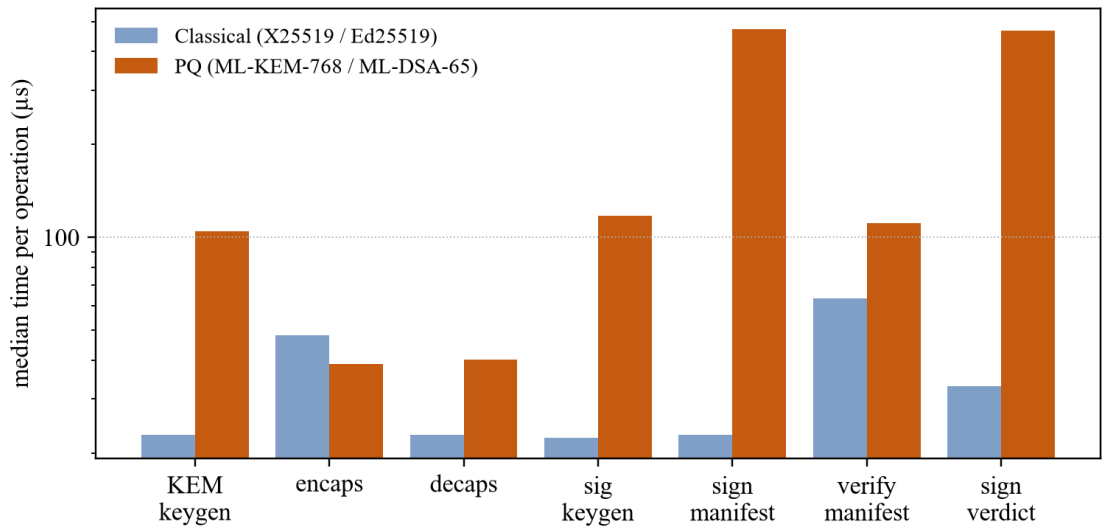


Fig. 3. Median time of each asymmetric operation, classical (X25519/Ed25519) vs post-quantum (ML-KEM-768/ML-DSA-65); logarithmic axis.

Table 9 and Fig. 3 show that ML-KEM-768 is as fast as X25519: encapsulation took 38.8 µs against 48.1 µs, and decapsulation 40.2 µs against 22.9 µs. ML-DSA-65 signing is the most expensive step at 473 µs median (1500 µs at p95, because its rejection sampling repeats a variable number of times), about 21 times Ed25519. Even so, every post-quantum operation stays below half a millisecond at the median, while sealing and unsealing the 2 MiB video take about 1.5 ms and 1.1 ms in both modes, dominated by AES-GCM over the payload rather than by the asymmetric algorithms.

Table 10. Key, ciphertext and signature sizes in bytes.

Mode	KEM public key	KEM ciphertext	Signature public key	Signature	Bundle overhead	Per-transaction asymmetric
Classical	32	32	32	64	505	96
PQ	1,184	1,088	1,952	3,309	4,799	4,397

The size cost is larger in relative terms: each transaction carries 4,397 bytes of KEM ciphertext and signature instead of 96 (Table 10). Against the 7.1 MB four-second video of the test session, the measured request grew by 4,287 bytes, 0.061% of its size.

4.3.2 End-to-end performance

Table 11. End-to-end results per mode (one recorded genuine session re-sent under fresh challenges, real models).

Metric	Classical	PQ
Runs	500	500
End to end p50 / p95 (ms)	3205.4 / 3493.4	3171.5 / 3293.0
Server total p50 / p95 (ms)	3176.8 / 3454.1	3140.5 / 3262.9
Cryptography p50 (ms)	23.194	26.409
Cryptography share of end to end (p50)	0.73%	0.83%

Metric	Classical	PQ
Bytes per request (mean)	7,069,357	7,073,644
Sequential throughput (requests/s)	0.34	0.31
CPU % mean / p95	682.3 / 930.9	665.0 / 831.7
Peak resident memory (MB)	1874.6	1876.2
Outcomes	rejected, MORPH_FLAG=500	rejected, MORPH_FLAG=500

Table 12. Latency per stage in milliseconds (p50 and p95).

Stage	Classical p50	Classical p95	PQ p50	PQ p95
client: challenge rtt	1.947	2.242	2.027	2.238
client: seal	18.172	23.087	19.331	24.193
client: verify rtt	3185.156	3466.124	3149.275	3271.399
server: parse	1.408	1.512	1.485	1.574
server: check1 challenge	0.446	0.538	0.449	0.495
server: check3 unseal	4.415	5.590	4.469	5.404
server: check4 binding	0.066	0.083	0.064	0.081
server: check5 hash	11.017	13.017	11.010	12.991
server: check6 signature	0.260	0.297	0.433	0.477
server: decode	15.813	19.418	15.192	17.789
server: faces	165.778	212.896	165.430	198.303
server: embed	942.593	1025.413	956.582	1021.242
server: det match	0.096	1.161	0.101	1.296
server: det m1 pad	66.406	87.791	64.533	82.257
server: det m2 morph	1.458	2.306	1.433	2.559
server: det m3 deepfake	2005.420	2218.822	1960.169	2045.019
server: det m4 replay	1132.835	1227.347	1113.227	1164.753
server: det m5 injection	21.407	30.122	21.420	26.849
server: detectors parallel	2007.124	2220.254	1962.013	2046.941
server: decision	0.006	0.007	0.006	0.007
server: verdict sign	0.226	2.706	1.986	3.437
server: crypto	4.954	7.564	6.946	8.999
server: total	3176.838	3454.089	3140.481	3262.899
end to end	3205.408	3493.364	3171.510	3293.045

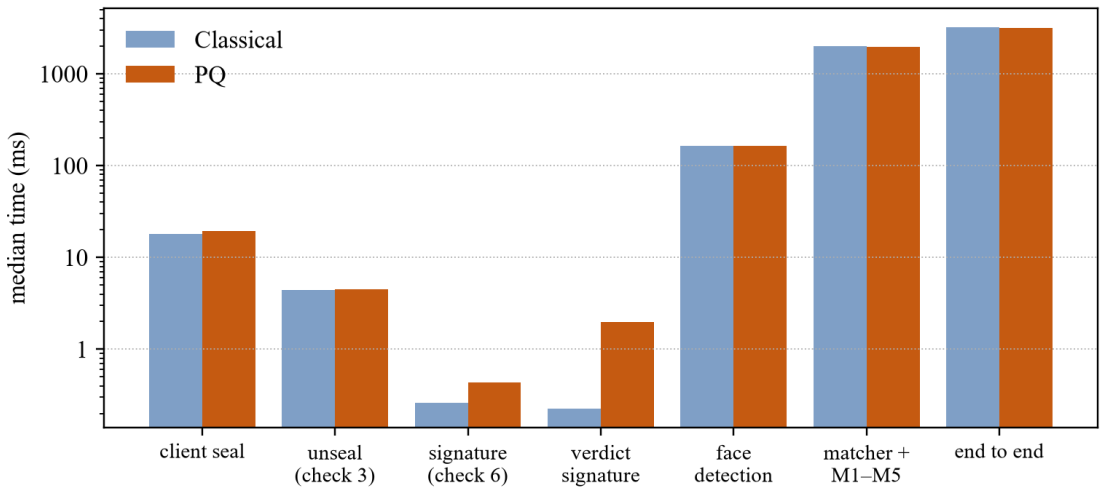


Fig. 4. Median latency of the main stages in both modes (logarithmic axis).

End to end, a transaction took a median of 3205 ms in classical mode and 3172 ms in PQ mode, a difference smaller than the run-to-run variation of the detectors; all cryptographic steps together took 23.2 ms and 26.4 ms, 0.73% and 0.83% of the total. The detectors dominate: the parallel matcher and M1–M5 stage took a median of 1962 ms in PQ mode, almost all of it the GenD vision transformer on three frames (1960 ms), and detecting and embedding the faces of the sampled frames took another 165 ms and 957 ms. The steps that change with the mode are small: the device-signature check took 0.26 ms classically and 0.43 ms with ML-DSA-65, and the verdict signature 0.23 ms and 1.99 ms; sealing and unsealing are dominated by AES-GCM over the video in both modes. The request grew from 7,069,357 to 7,073,644 bytes on average. CPU and memory were sampled from the single process that ran both client and server, so they describe the whole pipeline rather than the server alone. The run re-sends one recorded genuine session under fresh challenges and, having no separate enrolment photo, enrols the middle frame of that same recording as the reference. The "Outcomes" row records how every request was decided (rejected, MORPH_FLAG=500 in PQ mode): M2 was trained on studio enrolment photos paired with probes and did not accept this frame-from-the-same-video pairing. Every detector runs on every request whatever the decision, so the timings are unaffected, but this run is a cost measurement, not an accuracy result.

4.3.3 Protocol attacks: replay suite

Table 13. Replay suite.

Mode	Case	Result
both	all	pending

The 1,000-run replay suite with the real models had not finished when this report was generated.

4.3.4 Detection results

Face matcher. The threshold was chosen on the validation half of the FRLL subjects at an FMR of 0.1% and then applied unchanged to the other half.

Table 14. ArcFace matcher on FRLL (threshold from the validation half).

Half	Threshold	FMR	FNMR	Genuine pairs	Impostor pairs	Images without face
Validation	0.2192	0.094%	0.00%	255	12,750	0
Test	0.2192	0.102%	0.00%	255	12,750	0

On unseen subjects the FMR stayed at 0.102% (13 of 12,750 impostor pairs) and no genuine pair was rejected, including smiling and three-quarter views. FRLL is studio quality, so webcam images are harder (Section 4.3.6).

M2 morph detection. Results are pooled over the held-out folds of the 5-fold identity-grouped cross-validation.

Table 15. M2 (D-MAD) held-out APCER per morphing tool at three operating points; 1,020 bona fide pairs.

Target BPCER	Achieved BPCER	AMSL	FaceMorpher	OpenCV	StyleGAN	WebMorph
0.5%	0.59%	0.37%	0.22%	0.26%	0.00%	0.35%
1.0%	1.37%	0.27%	0.22%	0.13%	0.00%	0.22%
5.0%	5.29%	0.12%	0.00%	0.00%	0.00%	0.09%

At the deployed 0.5% target, 0.59% of genuine pairs were rejected and at most 0.37% of morphs were accepted for any tool; the EER was 0.31% and the AUC 0.9999. No image failed face detection. An earlier rule that set thresholds from the SVM's own training scores collapsed (training bona fide scores sit on the margin) and rejected 37% of genuine users on held-out data; it was replaced by the inner cross-validation, which is why these figures are not optimistic. The deployed threshold (0.5608) comes from the pooled held-out bona fide scores.

M3 deepfake detection. The headline comparison is on OpenForensics Test-Dev, which was never used for tuning or thresholds; both models use thresholds from the same 20% calibration split of Val.

Table 16. M3 (GenD) on OpenForensics Test-Dev: zero-shot vs LayerNorm-tuned.

Model	Target BPCER	Threshold	Achieved BPCER	APCER	EER	AUC
Zero-shot	0.5%	0.9734	0.43%	65.2%	12.61%	0.943
Zero-shot	1.0%	0.9640	0.90%	53.4%	12.61%	0.943
Zero-shot	5.0%	0.9092	5.26%	24.2%	12.61%	0.943
LayerNorm-tuned	0.5%	0.9182	0.21%	11.5%	1.75%	0.998
LayerNorm-tuned	1.0%	0.7555	1.39%	2.4%	1.75%	0.998
LayerNorm-tuned	5.0%	0.3588	5.06%	0.5%	1.75%	0.998

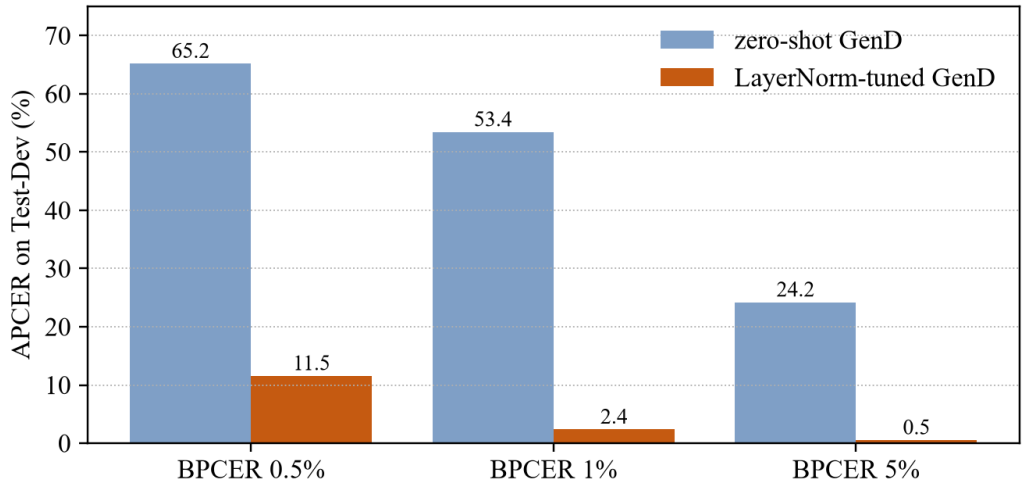


Fig. 5. Share of OpenForensics Test-Dev fakes accepted (APCER) at three genuine-rejection targets, zero-shot vs LayerNorm-tuned GenD.

Off the shelf, GenD ranks real and fake faces reasonably (AUC 0.943), but OpenForensics fakes differ from its training data and at a strict setting 65.2% of fakes pass. Tuning only the LayerNorm parameters for two epochs raised the AUC to 0.998 and reduced APCER to 11.5%, 2.4% and 0.5% at the three targets (Fig. 5). The achieved BPCER on Test-Dev (0.21%, 1.39%, 5.06%) stays close to the targets, which shows that thresholds from 769 calibration faces transfer; the tuned model is deployed.

Table 17. LayerNorm-only tuning of GenD, measured on the held-out 20% calibration split of Val.

Epoch	Training loss	Calibration AUC	Calibration EER
0	— (zero-shot)	0.9354	13.1%
1	0.2379	0.9885	4.2%
2	0.1122	0.9965	1.6%

Exported models. The server runs ONNX exports, so their outputs were compared with the original PyTorch models on 200 faces each.

Table 18. PyTorch vs ONNX parity of the exported models.

Module	Model	Faces	Minimum cosine	Max |difference|	Pass
M3	gend_clip_l14	200	0.9999998	4.6e-5	Yes
M1	2.7_80x80_MiniFASNetV2	200	0.9999998	1.8e-5	Yes
M1	4_0_0_80x80_MiniFASNetV1SE	200	0.9999998	1.6e-5	Yes
M3	gend_clip_l14_ln	200	0.9999998	8.4e-5	Yes

All exports match their originals with a minimum cosine similarity of 0.9999998. In addition, GenD's own pipeline and the server path (ONNX plus our preprocessing) differ by at most 5.19e-6 in P(fake) on 50 faces, with the same class for 50/50; the detector results above therefore apply to the running system.

4.3.5 Attack × mode matrix

Table 19. Attack × mode matrix.

Mode	Attack	Result
both	all	pending

The attack × mode matrix had not finished when this report was generated.

4.3.6 Live webcam test

Table 20. All genuine live captures of one team member at the laptop webcam, every capture logged.

Phase	Captures (PQ / classical)	Accepted	Rejected (reason)
First live check (uncalibrated thresholds)	1 / 1	1	1 (DEEPFAKE_FAIL ×1)
Lighting and position changed, old enrolment (zero-shot M3)	10 / 0	0	10 (MORPH_FLAG ×10)
Re-enrolled, zero-shot M3	5 / 5	9	1 (SCRIPT_FAIL ×1)
Re-enrolled, LayerNorm-tuned M3	5 / 5	9	1 (MORPH_FLAG ×1)

After re-enrolling in the same room, the user was accepted in 18 of 20 captures across both modes, with face-match scores of 0.79–0.88 on accepted captures (threshold 0.2192), a highest deepfake score of 0.35 and a highest presentation-attack score of 0.022. The two rejections were one SCRIPT_FAIL (the actions were not seen clearly in order) and one MORPH_FLAG with a low face match. The earlier phase shows the main practical risk: after the lighting and position changed, the old enrolment no longer matched and M2 flagged every capture. The system failed safe, but this is a usability cost.

4.3.7 Sources of error

In place of the quantum noise discussed in quantum-computing projects, the following sources of error affect these measurements:

- **Timing noise.** Measurements ran on a Windows laptop with turbo boost and background services; p95 values in particular vary between runs. ML-DSA signing time also varies by design because of rejection sampling.
- **Finite calibration sets.** A 0.5% target on 769 calibration faces leaves only three faces above the threshold, so the threshold itself is uncertain; Test-Dev BPCER of 0.21%–1.39% around targets of 0.5%–1% reflects this.
- **Dataset bias.** FRLL is London-based, OpenForensics and FRLL-Morphs are single sources, and all live captures come from one person and one webcam; results on other populations, cameras and morph tools are untested.
- **Numerical differences.** GenD on the GPU (TF32) differs from the CPU by up to 4.4×10^{-3} in P(fake); the server uses the CPU path, which matches GenD's own pipeline to 5×10^{-6}.

- **Environment drift.** Changes in lighting and pose between enrolment and verification lower the match score and can trigger M2 or M4.
- **Synthetic attack presentation.** In the attack matrix, still images are sent as static videos with webcam-like metadata, which is a simplification of a real injection.

5. Analysis and Discussion

5.1 Novelty and insights

- **A signed AI verdict.** Each decision is stored as an ML-DSA-65-signed record that binds the challenge, the transaction hash, the video hash, all scores, the model and threshold versions and the decision; editing any stored score makes verification fail, which the demonstration and the test suite show.
- **Post-quantum cryptography is cheap here.** ML-KEM-768 is as fast as X25519, and the slowest post-quantum step, ML-DSA-65 signing, takes about 0.47 ms at the median; end to end, cryptography is 0.83% of a PQ transaction. The real costs are the extra 4,301 bytes per transaction and, above all, the neural detectors.
- **Crypto and AI stop different attacks.** Binding checks stop replay, tampering, re-targeting and downgrade in both modes; presentation, morph and deepfake attacks pass every cryptographic check and are stopped only by the detectors. Post-quantum cryptography protects provenance and confidentiality, not genuineness.
- **Small tuning, large gain.** Training 0.03% of GenD's parameters cut accepted fakes at the strict operating point by a factor of about six on an independent test set.
- **Honest thresholds matter.** Setting M2 thresholds from the classifier's own training scores looked fine but rejected 37% of genuine users on held-out data; nested cross-validation fixed it.

5.2 Limitations

- **M1** uses the pretrained Silent-Face models with their own decision rule; fine-tuning and evaluation on CelebA-Spoof (including screen-replay types) are pending because the dataset requires a manual download.
- **M5** currently applies only the virtual-camera name rule; the gradient-boosted classifier needs 40–60 recorded sessions per class with OBS Studio, which is not yet installed.
- **Thresholds are preliminary** (version preliminary-val-2026-10-12-10537d5e) and were set on validation data; final calibration is planned.
- **Deployment** was tested on Windows with an in-process Redis replacement; the Docker image on Linux and a teammate's fresh-clone rebuild remain to be done, as do throughput tests under concurrent load.
- **Generalisation**: morph and deepfake results come from one dataset each, the matcher from studio photos, and the live tests from one person; performance on Indian faces and other webcams is untested.

- **Threat model**: a compromised server, side channels, denial of service, enrolment fraud, attackers on rooted devices and high-quality 3D masks are out of scope.

Table 21. Status of each part of the work at the time of this report.

Part	Status
Crypto suite, NIST known-answer tests, server checks 1–6, signed verdict	Done and tested
Matcher (FRLL), M2 (FRLL-Morphs), M3 zero-shot and tuned (OpenForensics Test-Dev)	Done, reported above
ONNX export parity (M1, M3)	Done
Crypto cost per operation and sizes	Done
End-to-end latency, CPU, memory, bytes	Done (500 runs per mode)
Replay suite with real models	Pending
Attack × mode matrix	Pending
M1 fine-tuning on CelebA-Spoof	Pending (dataset download)
M5 classifier on OBS recordings	Pending (OBS install, team recordings)
Final calibration, Docker on Linux, concurrent throughput, fresh-clone rebuild	Pending

6. Conclusions

This project built a working face-verification prototype in which every piece of evidence is bound to one transaction and one single-use challenge, protected with ML-KEM-768, ML-DSA-65 and AES-256-GCM, checked by six protocol checks and five AI detectors, and closed by a post-quantum-signed verdict that makes the AI decision auditable. The same pipeline runs with classical cryptography, which made it possible to measure the cost of post-quantum security directly.

The measurements answer the central question. Post-quantum cryptography is affordable for biometric evidence: its operations take well under a millisecond and add about 4.3 kB per transaction, a 0.83% share of the end-to-end time. But it only proves where the evidence came from and that it was not changed. The genuineness of the face depends on the detectors: the matcher met its 0.1% false-match target on unseen subjects, the morph detector accepted fewer than 4 in 1,000 morphs at a 0.6% genuine rejection, and a lightweight LayerNorm tuning made the deepfake detector usable at strict settings on an independent test set.

The remaining work is clear: fine-tune and evaluate M1 on CelebA-Spoof, record OBS sessions and train the M5 classifier, run the final calibration, and package the system in Docker so that a teammate can reproduce every result from a fresh clone. A natural extension is to publish the signed verdicts to an external transparency log, so that an auditor can check not only that a decision was not edited, but also that none was deleted.

Code

Source code (public): https://github.com/Aviralbhan/pq-biometric

The repository contains the full prototype, the tests, the evaluation scripts and the CSV files from which every number in this report is generated (docs/report/build_report.js and docs/report/make_figures.py). Datasets and model weights are not redistributed because of their licences; scripts/ downloads them from the original sources listed in Table 7 (FRLL: figshare 5047666; FRLL-Morphs: Zenodo 4415159; OpenForensics: Zenodo 5528418; CelebA-Spoof: its official repository). Commands for every step are in docs/RUNBOOK.md.

Originality Statement

We declare that this report and the accompanying software are our own work, carried out for this course project. The design of the protocol, the signed-verdict record, the evaluation protocol and all experiments were done by the team. Published algorithms, models, datasets and libraries are used under their licences and are cited where they are used; no text from other sources is reproduced without citation. All results come from our own runs and are traceable to CSV files in the repository.

AI Usage Statement

An AI coding assistant (Claude, by Anthropic) was used during the project to help write code, tests and documentation, to run and monitor experiments, and to draft this report from the team's design and result files. The team specified the design, reviewed the generated code and text, ran and checked the experiments, and verified that every number in the report is produced from the result CSVs by script. The AI tool was not used to generate or alter any result, and all references were checked against their sources. The team takes full responsibility for the content.

References

[1] ISO/IEC 30107-3:2023, *Information technology — Biometric presentation attack detection — Part 3: Testing and reporting*. Geneva, Switzerland: ISO, 2023.

[2] M. Ferrara, A. Franco, and D. Maltoni, "The magic passport," in *Proc. IEEE Int. Joint Conf. Biometrics (IJCB)*, 2014, pp. 1–7.

[3] R. Tolosana, R. Vera-Rodriguez, J. Fierrez, A. Morales, and J. Ortega-Garcia, "Deepfakes and beyond: A survey of face manipulation and fake detection," *Inf. Fusion*, vol. 64, pp. 131–148, 2020.

[4] D. Kurmankhojayev *et al.*, "Virtual camera detection: Catching video injection attacks in remote biometric systems," arXiv:2512.10653, 2025.

[5] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM J. Comput.*, vol. 26, no. 5, pp. 1484–1509, 1997.

[6] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proc. 28th Annu. ACM Symp. Theory Comput. (STOC)*, 1996, pp. 212–219.

[7] S. S. Gill *et al.*, "Quantum computing: A taxonomy, systematic review and future directions," *Softw. Pract. Exper.*, vol. 52, no. 1, pp. 66–114, 2022, doi: 10.1002/spe.3039.

[8] M. Mosca, "Cybersecurity in an era with quantum computers: Will we be ready?" *IEEE Secur. Privacy*, vol. 16, no. 5, pp. 38–41, 2018.

[9] Veridos GmbH, "The quantum risk in identity ecosystems: Managing the post-quantum migration window," white paper, 2026.

[10] National Institute of Standards and Technology, "Module-lattice-based key-encapsulation mechanism standard," FIPS 203, Aug. 2024, doi: 10.6028/NIST.FIPS.203.

[11] National Institute of Standards and Technology, "Module-lattice-based digital signature standard," FIPS 204, Aug. 2024, doi: 10.6028/NIST.FIPS.204.

[12] Daon, "Daon introduces five-patent Quantum Identity architecture for the next era of digital trust," press release, 2026 (covers U.S. Patent 12,695,590, issued Jul. 28, 2026). [Online]. Available: https://www.daon.com/resource/daon-introduces-five-patent-quantum-identity-architecture-for-the-next-era-of-digital-trust/

[13] C. Burt, "Post-quantum identity expands beyond cryptography," *Biometric Update*, Aug. 7, 2026. [Online]. Available: https://www.biometricupdate.com/202608/post-quantum-identity-expands-beyond-cryptography

[14] "authID adds post-quantum cryptography to biometric signature platform," *Biometric Update*, Apr. 2026. [Online]. Available: https://www.biometricupdate.com/202604/authid-adds-post-quantum-cryptography-to-biometric-signature-platform

[15] R. Román, R. Arjona, P. López-González, and I. Baturone, "A quantum-resistant face template protection scheme using Kyber and Saber public key encryption algorithms," in *Proc. Int. Conf. Biometrics Special Interest Group (BIOSIG)*, Darmstadt, Germany, 2022.

[16] R. Arjona, P. López-González, R. Román, and I. Baturone, "Post-quantum biometric authentication based on homomorphic encryption and Classic McEliece," *Appl. Sci.*, vol. 13, no. 2, Art. 757, 2023, doi: 10.3390/app13020757.

[17] A. Olushola and S. P. Meenakshi, "A post-quantum privacy-preserving multimodal biometric authentication framework integrating ML-KEM (Kyber) and fully homomorphic encryption," *Front. Phys.*, vol. 14, Art. 1849257, 2026, doi: 10.3389/fphy.2026.1849257.

[18] N. S. Gowri Ganesh, J. Jesila Mol, S. Manoj Kumar, V. Reddy, and T. Meyyappan, "Biometric spoofing detection with post-quantum cryptography integration," in *Proc. 9th Int. Conf. Comput. Intell. Data Sci. (ICCIDS)*, Chennai, India, Jan. 2026, doi: 10.1109/ICCIDS69108.2026.11407868.

[19] Y. Zhang *et al.*, "CelebA-Spoof: Large-scale face anti-spoofing dataset with rich annotations," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2020, pp. 70–85.

[20] Minivision AI, "Silent-Face-Anti-Spoofing," GitHub repository, 2020. [Online]. Available: https://github.com/minivision-ai/Silent-Face-Anti-Spoofing

[21] D. Tang, Z. Zhou, Y. Zhang, and K. Zhang, "Face Flashing: A secure liveness detection protocol based on light reflections," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2018.

[22] G. Mittal, C. Hegde, and N. Memon, "GOTCHA: Real-time video deepfake detection via challenge-response," arXiv:2210.06186, 2023.

[23] U. Scherhag, C. Rathgeb, J. Merkle, and C. Busch, "Deep face representations for differential morphing attack detection," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 3625–3639, 2020.

[24] E. Sarkar, P. Korshunov, L. Colbois, and S. Marcel, "Vulnerability analysis of face morphing attacks from landmarks and generative adversarial networks," arXiv:2012.05344, 2020.

[25] Idiap Research Institute, "FRLL-Morphs," Zenodo record 4415159. [Online]. Available: https://zenodo.org/record/4415159

[26] T.-N. Le, H. H. Nguyen, J. Yamagishi, and I. Echizen, "OpenForensics: Large-scale challenging dataset for multi-face forgery detection and segmentation in-the-wild," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2021, pp. 10117–10127.

[27] A. Radford *et al.*, "Learning transferable visual models from natural language supervision," in *Proc. 38th Int. Conf. Mach. Learn. (ICML)*, 2021, pp. 8748–8763.

[28] A. Yermakov, J. Cech, J. Matas, and M. Fritz, "Deepfake detection that generalizes across benchmarks," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis. (WACV)*, 2026, doi: 10.1109/WACV61042.2026.00082.

[29] A. Langley, M. Hamburg, and S. Turner, "Elliptic curves for security," RFC 7748, Jan. 2016.

[30] S. Josefsson and I. Liusvaara, "Edwards-curve digital signature algorithm (EdDSA)," RFC 8032, Jan. 2017.

[31] National Institute of Standards and Technology, "Secure hash standard (SHS)," FIPS 180-4, Aug. 2015, doi: 10.6028/NIST.FIPS.180-4.

[32] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC," NIST SP 800-38D, Nov. 2007.

[33] O. Regev, "On lattices, learning with errors, random linear codes, and cryptography," *J. ACM*, vol. 56, no. 6, Art. 34, 2009.

[34] A. Langlois and D. Stehlé, "Worst-case to average-case reductions for module lattices," *Des. Codes Cryptogr.*, vol. 75, no. 3, pp. 565–599, 2015.

[35] J. Bos *et al.*, "CRYSTALS – Kyber: A CCA-secure module-lattice-based KEM," in *Proc. IEEE Eur. Symp. Secur. Privacy (EuroS&P)*, 2018, pp. 353–367.

[36] L. Ducas *et al.*, "CRYSTALS-Dilithium: A lattice-based digital signature scheme," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2018, no. 1, pp. 238–268, 2018.

[37] National Institute of Standards and Technology, "ACVP-Server: Automated Cryptographic Validation Protocol," GitHub repository. [Online]. Available: https://github.com/usnistgov/ACVP-Server

[38] H. Krawczyk and P. Eronen, "HMAC-based extract-and-expand key derivation function (HKDF)," RFC 5869, May 2010.

[39] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," RFC 2104, Feb. 1997.

[40] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive angular margin loss for deep face recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 4690–4699.

[41] J. Guo, J. Deng, *et al.*, "InsightFace: 2D and 3D face analysis project," GitHub repository. [Online]. Available: https://github.com/deepinsight/insightface

[42] W. Wu, H. Peng, and S. Yu, "YuNet: A tiny millisecond-level face detector," *Mach. Intell. Res.*, vol. 20, no. 5, pp. 656–665, 2023.

[43] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[44] C. Lugaresi *et al.*, "MediaPipe: A framework for building perception pipelines," arXiv:1906.08172, 2019.

[45] G. Ke *et al.*, "LightGBM: A highly efficient gradient boosting decision tree," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 3146–3154.

[46] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019.

[47] Python Cryptographic Authority, "cryptography," version 50.0.2. [Online]. Available: https://cryptography.io

[48] ONNX Runtime developers, "ONNX Runtime," version 1.31.0. [Online]. Available: https://onnxruntime.ai

[49] L. DeBruine and B. Jones, "Face Research Lab London Set," figshare, 2017, doi: 10.6084/m9.figshare.5047666.

Appendix A. Reason codes and reproduction

Table 22. Server reason codes (kept in the server log; the client only sees accepted / rejected).

Code	Meaning
EXPIRED	Challenge lifetime (60 s) passed
REPLAY	Challenge already consumed
MODE_MISMATCH	Bundle mode differs from the challenge mode
TAMPER	KEM decapsulation or AES-GCM authentication failed
TX_MISMATCH	Nonce, transaction hash, challenge id or mode differ
HASH_MISMATCH	Video hash differs from the signed manifest
SIG_INVALID	Device signature invalid
NO_MATCH	Face match below threshold
PAD_FAIL	M1 presentation attack
MORPH_FLAG	M2 morphed reference
DEEPFAKE_FAIL	M3 deepfake
SCRIPT_FAIL	M4 actions not performed in order
INJECTION_FAIL	M5 virtual camera / injection

Reproduction order (from docs/RUNBOOK.md): install requirements; run pytest; download the datasets and models with the scripts in scripts/; run eval_matcher.py, train_m2.py, eval_detectors.py and the M3 tuning; python -m eval.crypto_bench; python -m eval.perf --runs 500; scripts/replay_attacks.py --runs 1000; python -m eval.attack_matrix; then docs/report/make_figures.py and docs/report/build_report.js.